Responsible AI, Safety and Evaluation on Azure AI Foundry

Azure AI Foundry guide, volume 3 · Version 2026.1

> Sample foundation document written for testing a domain-expert RAG. Azure AI Foundry evolves quickly and product names have changed over time; confirm details in the official Microsoft Learn documentation.

1. Microsoft's Responsible AI principles

Microsoft's Responsible AI Standard is built on six principles: fairness; reliability and safety; privacy and security; inclusiveness; transparency; and accountability. Foundry's safety and evaluation features are the practical tools for meeting them: map risks, measure them, mitigate them and operate with monitoring.

2. Azure AI Content Safety

Capability	What it detects
Harm categories	Hate, sexual, violence and self-harm content in text and images, each scored by severity
Prompt Shields	User prompt attacks (jailbreaks) and document attacks (indirect prompt injection hidden in documents, emails or tool results)
Groundedness detection	Claims in a response that are not supported by the provided source documents
Protected material detection	Known copyrighted text such as song lyrics and articles, and public code
Custom categories and blocklists	Organisation-specific terms and topics

3. Prompt Shields in practice

Prompt Shields analyse two inputs separately. The **user prompt** is checked for jailbreak attempts such as role play, attempts to change system rules or encoded instructions.

Each **document** (retrieved passages, emails, web content, tool results) is checked for embedded instructions aimed at the model, the typical sign of indirect prompt injection. Send retrieved content as documents rather than concatenating it into the prompt so the shield can inspect it. If an attack is detected, drop or quarantine that document, log the event, and answer from the remaining sources. Prompt Shields are one layer; still delimit untrusted content, limit tool privileges and require approval for sensitive actions.

4. Content filters on deployments

Every model deployment in Foundry has a content filter configuration applied to prompts and completions. Default configurations block medium and high severity harmful content; custom configurations can adjust thresholds per category, enable Prompt Shields, protected material detection and groundedness features, and attach blocklists. Changes to reduce filtering for sensitive use cases may need approval.

5. Evaluation in Foundry

Evaluator family	Examples	Measures
AI-assisted quality	Groundedness, relevance, coherence, fluency, similarity, retrieval	Scored by a judge model, typically on a 1 to 5 scale
NLP metrics	F1 score, BLEU, ROUGE, METEOR	Overlap with reference answers
Risk and safety	Hateful and unfair, sexual, violent, self-harm, protected material, indirect attack jailbreak	Rate of unsafe responses, usually measured on adversarial data
Agent evaluators	Intent resolution, tool call accuracy, task adherence	Whether an agent understood the task and used tools correctly
Custom	Your own code or prompt-based evaluators	Domain rules, formats, citations

6. Evaluating groundedness of a RAG app

1. Build a test dataset with query, response, and the context (retrieved passages) for

each row, plus ground-truth answers where available.
2. Run the groundedness evaluator, which checks whether each claim in the response is supported by the context, alongside relevance and retrieval evaluators.
3. Inspect low-scoring rows: a grounded but irrelevant answer usually means retrieval failed; an ungrounded answer means the model added unsupported claims.
4. Fix retrieval (chunking, hybrid search, semantic ranker, filters) or the prompt, and re-run.
5. Add the evaluation to CI and to continuous monitoring on sampled production traffic.

```
from azure.ai.evaluation import GroundednessEvaluator, evaluate

groundedness = GroundednessEvaluator(model_config)
result = evaluate(data="rag_testset.jsonl", evaluators={"groundedness": grounded
```

7. Red teaming and adversarial simulation

Foundry provides adversarial simulators that generate attack conversations against your application, and an AI red teaming agent built on the open-source PyRIT framework that runs automated attack strategies (for example encoding, character manipulation and jailbreak styles) and reports an attack success rate by risk category. Run these before release and after every model or prompt change, and combine them with manual red teaming for domain-specific risks.

8. Governance checklist

Area	Practice
Inventory	Record each AI app, its models, data sources, owner and risk level
Access	Entra ID and managed identities; least-privilege roles; no shared keys
Network	Private endpoints and managed network egress rules
Safety	Content filters with Prompt Shields and groundedness enabled
Quality	Evaluation gates in CI; monitoring in production
Transparency	Tell users they are using AI; show sources for answers

Incidents	Log prompts and outputs with retention rules; a process to disable features quickly